



## TOPIC-THREE

# Git Basics

Author: Arash Foroughi



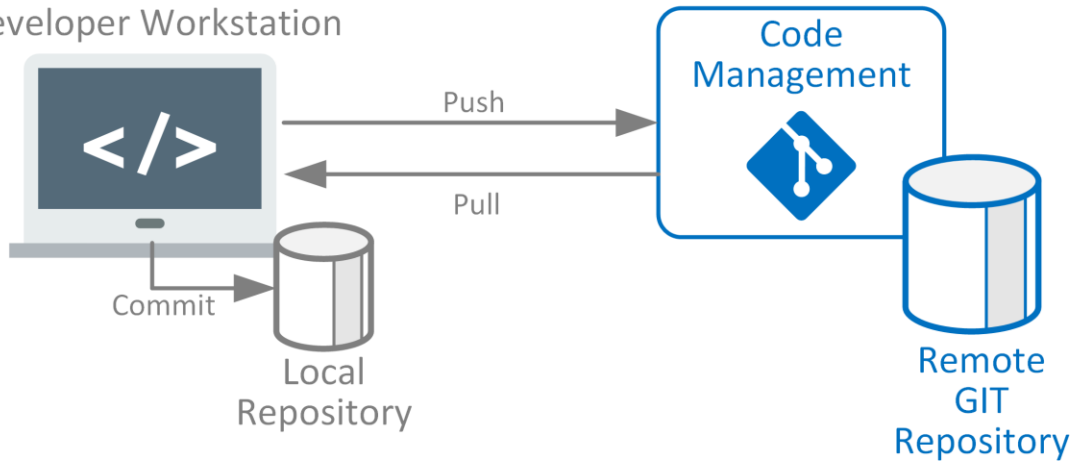


# Git Introduction



# Version Control System (VCS)

Developer Workstation



Track History

Work Together

## BENEFITS OF VERSION CONTROL

01



Streamline merging and branching

02



Examine and experiment with code

03



Discover the ability to operate offline

04



Create regular, automated backups

05

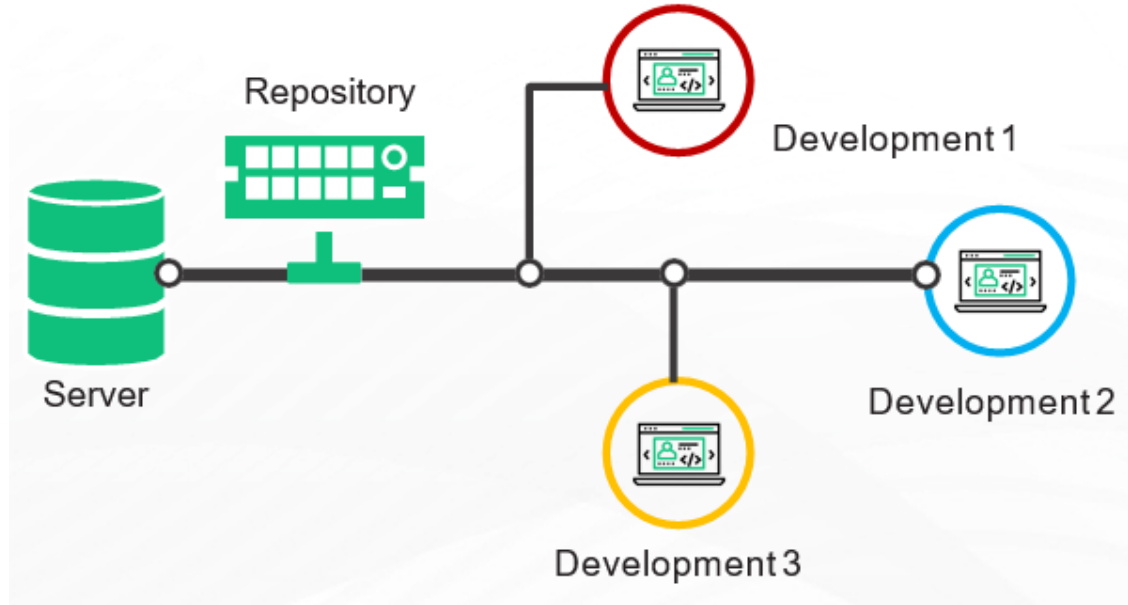


Communicate through open channels

# "Centralized" VCS

- **CVCS** is a type of software tool that helps developers manage the **changes** and versions of their code files in a central server.
- In a CVCS, every user **commits** their changes **directly** to the **main branch** of the project.

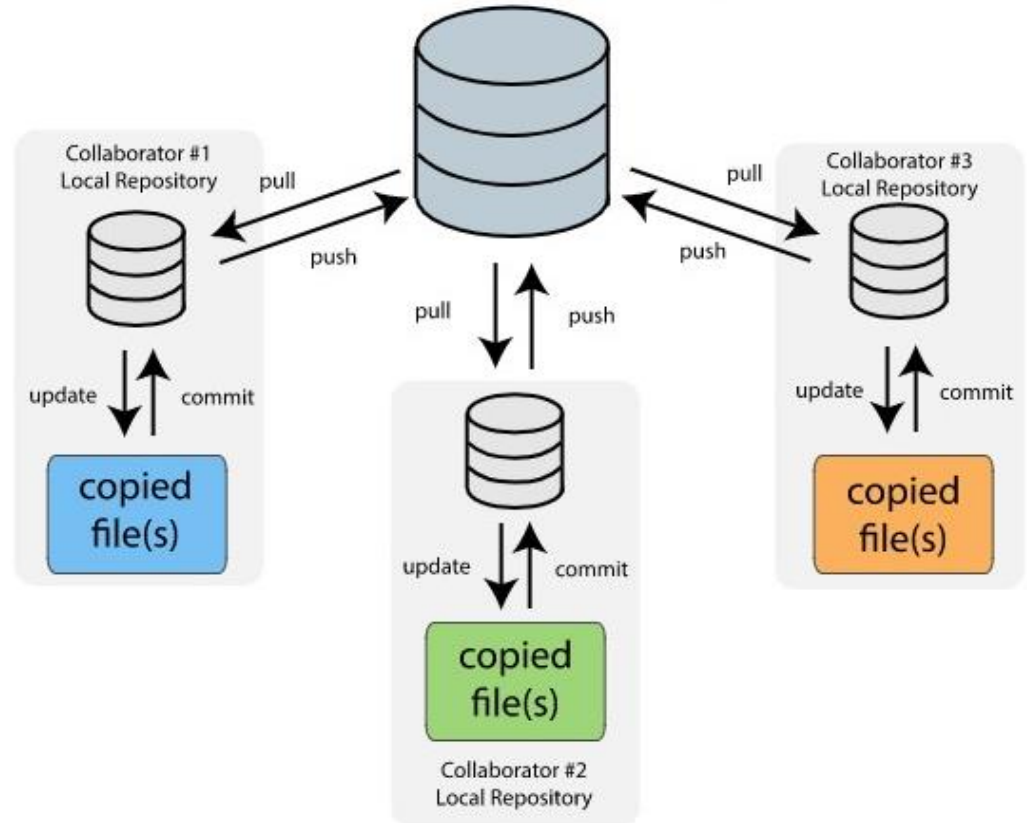
CVS, Perforce, Subversion (SVN),  
Microsoft Team Foundation Server (TFS)



# "Distributed" VCS

- **DVCS** is a type of software tool that helps developers manage the **changes** and **versions** of their code files in a peer-to-peer manner.
- In a DVCS, every user has a **local copy** of **entire code** and its **history**, and can work on it **independently** without relying on a central server.

Git, Mercurial, Bazaar



# Source Code Management (SCM)

- It's a method to **keep** a software system that consists of **versions/configs**, well organized.
- There are many **Version Control Systems**, such as:
  - **CVS** - Kind of the grandfather of version control
  - **PVCS** - Commercialized CVS
  - **Subversion** - inspired by CVS
  - **Perforce**
  - **Microsoft Visual SourceSafe**
  - **Mercurial**
  - **TeamSite**
  - **Vault**
  - **Bitkeeper** - Used to manage the Linux kernel before...
  - **Git** - Created by our favorite Linux author & creator: Linus!

Global  
Information  
Tracker

<https://git-scm.com/>

# Git Installation

# Installing Git on Linux

## ❑ Installing git on Linux servers:

```
root@host:~# apt install git
root@host:~# git --version
git version 2.30.2
```

## ❑ Global username is because to identify yourself when making changes on repositories.

```
root@host:~# git config --global user.name "Arash Foroughi"
root@host:~# git config --global user.email "arash@localhost"
root@host:~# cat ~/.gitconfig
```

# Git Basic Configurations

## Local Configuration

```
$ cat .git/config
```

Local configs are only available for the current **project** and stored in `.git/config` in the project's directory.

## Global Configuration

```
$ cat ~/.gitconfig
```

Global configs are available for **all projects for the current user** and stored in `~/.gitconfig`.

## System-level Configuration

```
$ cat /etc/gitconfig
```

System config applies to the entire system for all users & projects and stored in `/etc/gitconfig`.



# Git Basic Configurations

```
root@host:~# git config --system system.name "git repo server-1"
root@host:~# git config --system user.name "Arash Foroughi"
user@host:~$ git config --global system.name "my repo server-1"
user@host:~$ git config --global core.editor vim
user@host:~$ git config --global core.pager 'less'
user@host:~$ git config --list
user@host:~$ cat ~/.gitconfig
[user]
    name = Root User
    email = root@localhost
[system]
    name = git repo server-1
[core]
    editor = vim
    pager = more
```

# Empty Repository

# Create Our First Local Git Repo

❑ **STEP-1:** Use git init command to initialize a location as git repository:

```
arash@host:~$ mkdir gittest
```

```
arash@host:~$ cd gittest
```

```
arash@host:~$ git init
```

```
Initialized empty Git repository in /home/ubuntu/gittest/.git/
```

```
arash@host:~$ cd .git
```

```
arash@host:~$ ls -l
```

```
total 32
```

```
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 30 21:25 branches
```

```
-rw-rw-r-- 1 ubuntu ubuntu  92 Sep 30 21:25 config
```

```
-rw-rw-r-- 1 ubuntu ubuntu  73 Sep 30 21:25 description
```

```
-rw-rw-r-- 1 ubuntu ubuntu  23 Sep 30 21:25 HEAD
```

```
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 30 21:25 hooks
```

```
drwxrwxr-x 2 ubuntu ubuntu 4096 Sep 30 21:25 info
```

# Create Our First Local Git Repo

❑ **STEP-2:** Create files in repo needed to be tracked by git and then use git add:

```
arash@host:~$ git status
```

```
On branch master
```

```
No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
```

```
arash@host:~$ echo "This is my first file" > test.txt
```

```
arash@host:~$ git add test.txt or git add * or git add -A
```

```
arash@host:~$ git status
```

```
On branch master
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   test.txt
```

# Create Our First Local Git Repo

❑ **STEP-1:** Use git commit to finalize your changes as the last version:

```
arash@host:~$ git commit -m "This is my first git file"
[master (root-commit) e74c7da] This is my first git file
1 file changed, 1 insertion(+)
create mode 100644 test.txt
```

```
arash@host:~$ git status
On branch master
nothing to commit, working tree clean
```



# Git Basics



# Git Basics – Setup, Init, Stage & Commit

- Initializing repositories and working with **Git staging area**:

| Git Commands                   | Description                                                       |
|--------------------------------|-------------------------------------------------------------------|
| <code>Git init</code>          | Initialize an existing directory as a Git Repository.             |
| <code>Git status</code>        | Show modified files in working directory, staged for next commit. |
| <code>Git add [FILE]</code>    | Add a file as it looks now to your next commit (stage).           |
| <code>Git reset [FILE]</code>  | Unstage a file while retaining the changes in working directory.  |
| <code>Git diff HEAD</code>     | Show difference between working directory and last commit (HEAD). |
| <code>Git diff --staged</code> | Diff of what is staged but not yet committed.                     |
| <code>Git diff CM1 CM2</code>  | Show the differences between two commits.                         |
| <code>Git commit</code>        | Commit your staged content as a new commit snapshot.              |

# Git Basics

## ❑ Example of adding & removing files to a git repository:

```
arash@host:~$ mkdir source; cd source; tar -zxvf source.tar.gz
arash@host:~$ git init; git status
arash@host:~$ git add *
arash@host:~$ git commit -m "Initial source code check"
arash@host:~$ git status
arash@host:~$ rm TODO
arash@host:~$ git status
arash@host:~$ git rm TODO
arash@host:~$ git commit -m "Deleting TODO"
arash@host:~$ git status
```

# Using Git Ignore to not Track a File

❑ For example you don't want to track README file in the source code:

```
arash@host:~$ git init
arash@host:~$ vim .gitignore : README
arash@host:~$ git add *
arash@host:~$ git commit -m "All Except README"
arash@host:~$ git status

On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .gitignore

nothing added but untracked files present (use "git add" to track)
```

# Cloning Git Repositories

# Cloning LOCAL Repositories

- ❑ Physical filesystem full representative copy of the original repository.
- ❑ NOTICE: It's just a copy of your repository & it's not linked to your original repository!

```
arash@host:~$ cp -rf gittest gittest2
```

```
arash@host:~$ cd gittest2
```

```
arash@host:~$ git status
```

```
On branch master
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
.gitignore
```

```
nothing added but untracked files present (use "git add" to track)
```

# Cloning LOCAL Repositories

- ❑ Retrieve an entire copy of repository from a local repository.
- ❑ NOTICE: It's a copy of your repository which it is linked to the original repository!

```
arash@host:~$ git clone /home/ubuntu/gittest gittest2
arash@host:~$ cd gittest2
arash@host:~$ vi testfile.txt : "New File Added"
arash@host:~$ git status; git add testfile.txt
arash@host:~$ git commit -m "New File Added"
arash@host:~$ cd ../gittest
arash@host:~$ git pull /home/ubuntu/gittest2
```

# Cloning REMOTE Repositories

- ❑ Retrieve an entire copy of repository from a remote repository.

```
arash@host:~$ ssh-keygen
arash@host:~$ ssh-copy-id arash@192.168.1.2
arash@host:~$ git clone arash@192.168.1.2:~/gittest gittest_local
arash@host:~$ cd gittest_local; vi new_file --> "new file added"
arash@host:~$ git add new_file
arash@host:~$ git commit -m "New file added"
arash@host:~$ git pull root@192.168.1.3:~/gittest_local

* branch              HEAD          -> FETCH_HEAD
Fast-forward
new_file | 1 +
1 file changed, 1 insertion(+)
create mode 100644 new_file
```

# Git Tags

# Tags (It's like a Snapshot!)

- A **tag** is created to **point** to a **very specific commit** and **doesn't change** never.
- They're **not really snapshots**, but they are as good as a **snapshot-based database**.
- It's just a **pointer** to a **specific commit** or a place in a time.
- There are two flavors of tags ---> **Annotated & Non-Annotated**
  - **Annotated Tag** has a message that can be displayed with git show.
  - **Non-Annotated Tag** is just a named pointer to a commit without any message.

Annotated Tag

Non-Annotated Tag

# Creating a Non-Annotated Tag

- ❑ Here is an example of how to **create & display** a **Non-Annotated** Tag:

```
arash@host:~$ git tag tag1
arash@host:~$ git tag
tag1
arash@host:~$ vi file1.txt; git add -A; git commit -m "Tag Added"
arash@host:~$ git show tag1
Author: Arash Foroughi <arash@localhost.localdomain>
Date: Thu Aug 14 15:10:32 2021 +0330
diff --git a/new_file b/new_file
new file mode 100644
index 0000000..dfc8289
--- /dev/null
+++ b/new_file
@@ -0,0 +1 @@
+new added
arash@host:~$ git describe --tags
tag1-1-g4ff7c1b
```

# Creating an Annotated Tag

- ❑ Here is an example of how to **create & display** a **Annotated** Tag:

```
arash@host:~$ git tag -a V1 -m "Version 1.0 Released"
```

```
arash@host:~$ git tag
```

```
V1
```

```
arash@host:~$ vi file1.txt; git add -A; git commit -m "Tag1 added"
```

```
arash@host:~$ git show V1
```

```
Tagger: "Arash <"arash@localhost">"
```

```
Version 1.0 Released
```

```
commit 4ff7c1b2422d8166d4fb376bb48e1279f08bc999 (tag: V1)
```

```
Author: "Arash <"arash@localhost">"
```

```
Tag1 Added
```

```
diff --git a/file1.txt b/file1.txt
```

```
index 0000000..cd08755
```

```
--- /dev/null
```

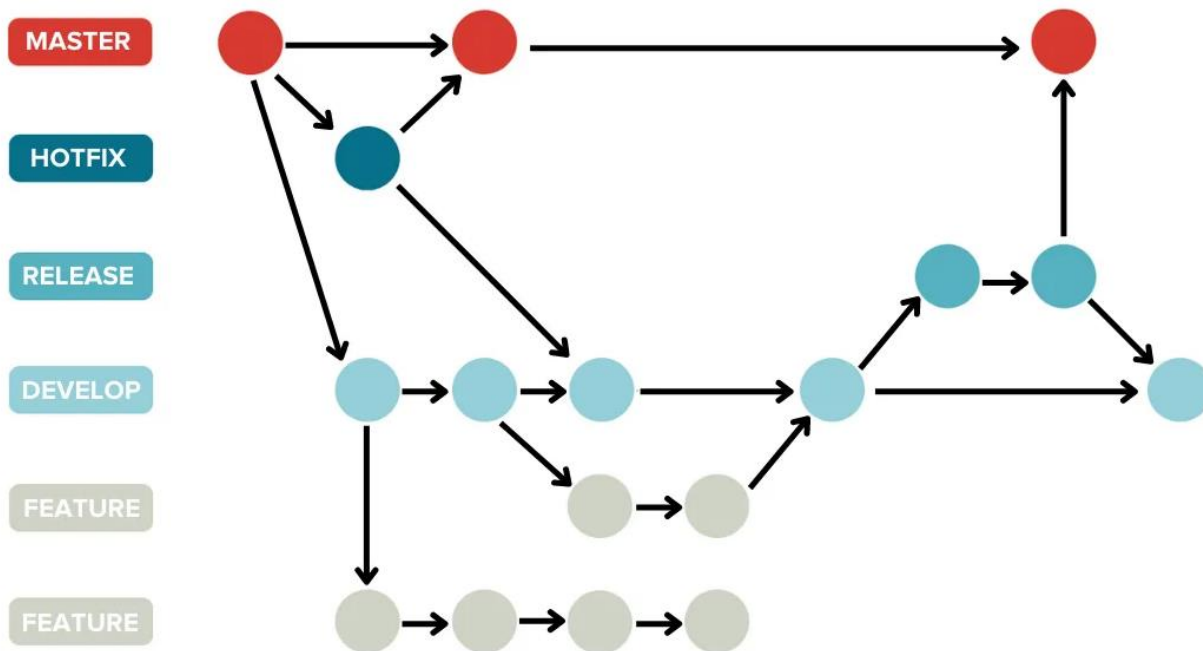
```
+++ b/file1.txt
```

```
@@ -0,0 +1 @@
```

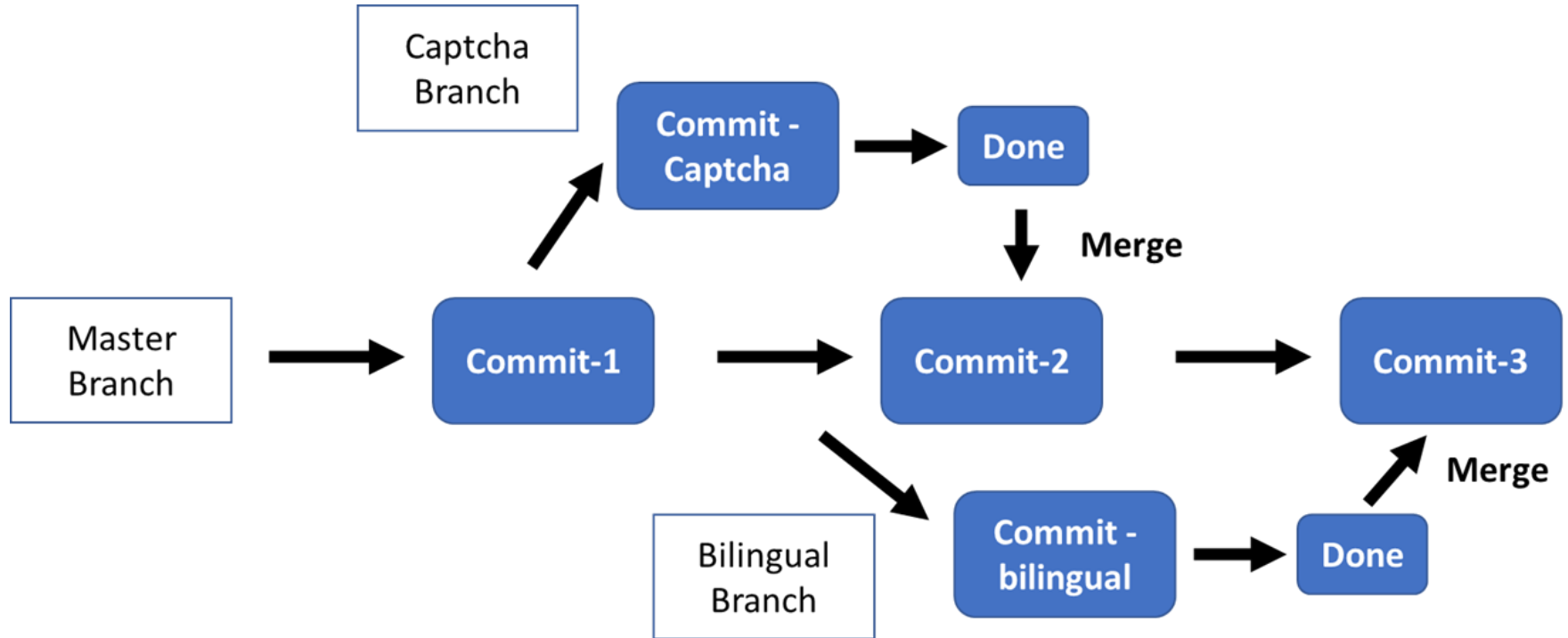
# Git Branches

# Git Branches

- A branch in Git is simply a **lightweight movable pointer to one of the commits**.
- **Master branch** always points to the **last commit** you made on.



# What is Git Branch?



# Git Branch Management

- Isolating work in **branches**, **changing context**, and **integrating** changes:

| Git Commands                            | Description                                                      |
|-----------------------------------------|------------------------------------------------------------------|
| <code>Git branch -a</code>              | List all of your branches.                                       |
| <code>Git branch [BRANCH]</code>        | Creating a new branch at the current commit.                     |
| <code>Git checkout [BRANCH]</code>      | Switch to another branch.                                        |
| <code>Git checkout -b [BRANCH]</code>   | Create a new branch and switch to it at the moment.              |
| <code>Git merge [BRANCH]</code>         | Unstage a file while retaining the changes in working directory. |
| <code>Git merge [BRANCH] --no-ff</code> | Create a merge commit in all cases. No fast-forward.             |
| <code>Git log</code>                    | Show all commits in the current branch's history.                |

# Creating & Switching to a New Branch

❏ Here is an example of creating a new branch called "Development":

```
arash@host:~$ git branch -a
arash@host:~$ git checkout -b development
arash@host:~$ git branch development --> (Creating New Branch)
arash@host:~$ git checkout development --> (Switching to New Branch)
arash@host:~$ vi development.txt : "test of development branch"
arash@host:~$ git add development.txt
arash@host:~$ git commit -m "development test commit"
arash@host:~$ git status
    On branch development
    nothing to commit, working tree clean
arash@host:~$ git checkout master
```

# Merging Two Branches Together

## ❑ How to merge development branch with master branch:

### I. First, switch to Master Branch:

```
arash@host:~$ git checkout master  
Switched to branch 'master'
```

### II. Then, merge Development branch in Master branch:

```
arash@host:~$ git merge development --no-ff  
Updating 3c1599c..5292e50  
Fast-forward  
development.txt | 1 +  
1 file changed, 1 insertion(+)  
create mode 100644 development.txt
```

# Git Log

# Git Log

❑ How to see the latest logs of commits with git log command:

```
arash@host:~$ git log
```

```
arash@host:~$ git log -p -2
```

```
arash@host:~$ git log --stat
```

```
arash@host:~$ git log --pretty=oneline
```

```
arash@host:~$ git log --pretty=oneline --graph
```

```
arash@host:~$ git log --pretty=reference
```

```
arash@host:~$ git log --graph
```

```
arash@host:~$ git log --graph --pretty=format:'%Cred%h%Creset -  
%C(yellow)%d%Creset %s %Cgreen(%cr) %C(bold blue)<%an>%Creset' --  
abbrev-commit
```

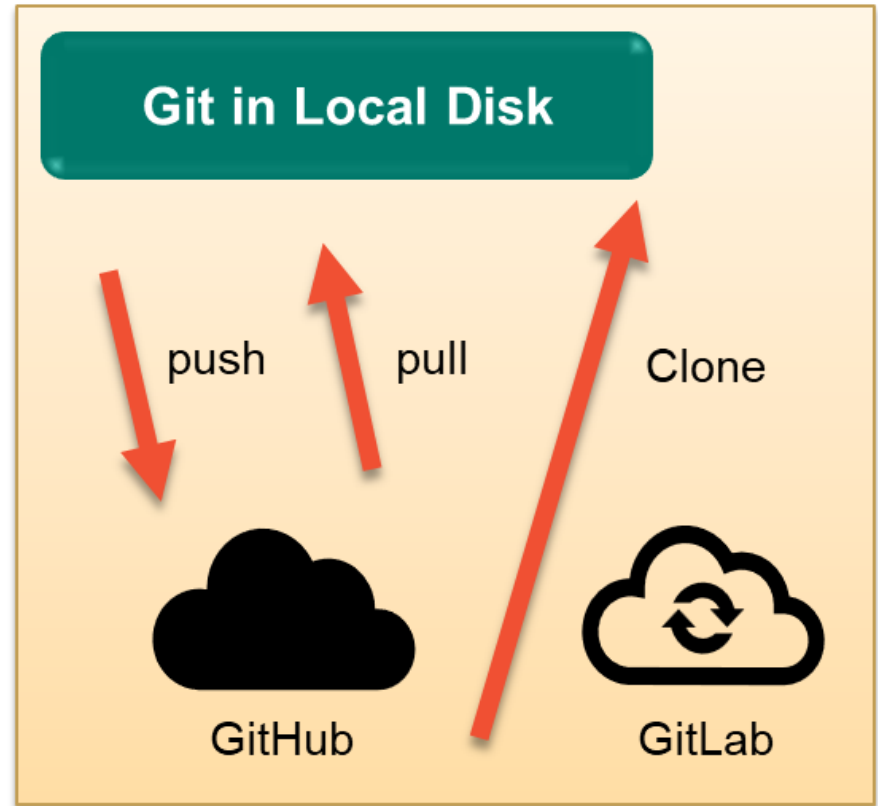


# Introduction to GitHub & GitLab



# Introduction to GitHub

- **GitHub** is designed to help people better **manage open-source projects**.
- The open source version control software that lets **multiple people** make **separate changes** to codes at the same time.
- GitHub has **free public account**, but for **private** repos, should pay monthly & it's **not free**.



# How to Push to GitHub?

- After creating GitHub account, you can create a Repository on it, and **push your local repository to GitHub** or **pull repository from GitHub** to your local repository.
- ❑ **Create Personal Access Token on GitHub: (from 13 August 2021)**
  - From your GitHub account, go to **Settings => Developer Settings => Personal Access Token => Generate New Token** (Give your password) => **Fill-up the form** => click **Generate token** => **Copy the generated Token**, it will be something like below:
  - `ghp_sFhFsSHhTzMDreGRLjmks4Tzuzgthdvfsrta`
- Use that **token** as your **password** when you want to **push/pull** or **clone** something.

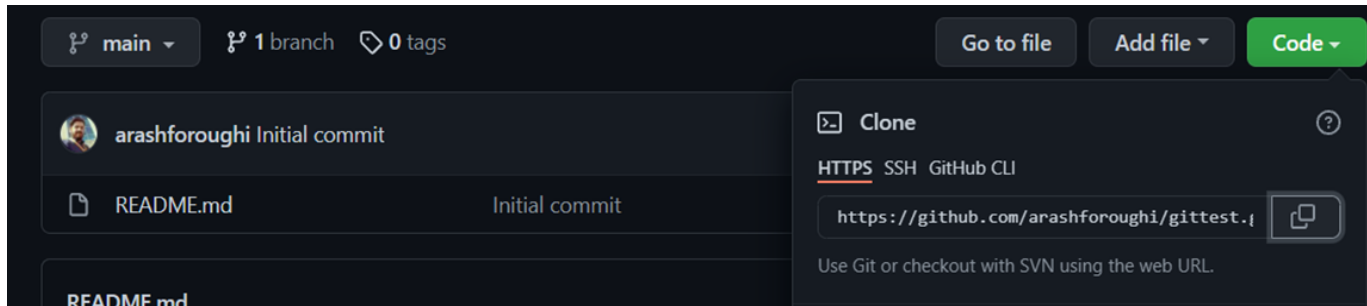
# Git Branch Management

- Isolating work in **branches**, changing context, and **integrating** changes:

| Git Commands                              | Description                                                     |
|-------------------------------------------|-----------------------------------------------------------------|
| <code>Git remote add [ALIAS] [URL]</code> | Add a git URL as an alias.                                      |
| <code>Git fetch [ALIAS]</code>            | Fetch down all branches from the Git remote.                    |
| <code>Git fetch [ALIAS] [BRANCH]</code>   | Fetch down a specific branch from the Git remote.               |
| <code>Git pull [ALIAS] [BRANCH]</code>    | Fetch down the specified remote's copy to the specified branch. |
| <code>Git push [ALIAS] [BRANCH]</code>    | Push the branch updates to the Git remote.                      |

# Push a Local Repo to GitHub

- For **pushing** your **Local Repo** to a **GitHub Repository**, do below steps:
  - 1) First, **create a repository** in your GitHub and copy its **HTTPS** address:



- 2) Then, in your **local repo**, use below command to **push it to you GitHub Repo**.

```
arash@host:~$ git remote add [alias] https://github.com/arash/repo.git
arash@host:~$ git remote -v
arash@host:~$ git push -u origin master
```

# Clone or pull a Repo from GitHub

- For **Cloning** a **GitHub Repo** to your **Local Git Repository**, do below steps:

1) **Copy SSH or HTTPS code and use `git clone` command:**

```
arash@host:~$ git clone https://github.com/arashforoughi/repo.git
arash@host:~$ cd gittest
arash@host:~$ vi README : change something
arash@host:~$ git add -A
arash@host:~$ git commit -m "test"
arash@host:~$ git push origin master
```

2) **If you changed something in GitHub editor, you can use `git pull` command.**

```
arash@host:~$ git pull origin master
```

# Introduction to GitLab



**GitLab** is **open-source** for community edition. **GitHub** is not **open-source**.



GitLab has **free private repositories**, but GitHub does not have.



GitHub allows users to make unlimited free **public repository**.



In GitLab, addition to **create repositories**, you can also **make CI/CD pipelines** using Docker, NPM and other softwares, like something we will do with **Jenkins**.



Working with GitLab in repositories is exactly same as working with GitHub.

# Installing GitLab-CE Locally

## ❑ STEP-1. Install Dependencies:

```
root@host:~# apt install curl openssh-server ca-certificates postfix
```

## ❑ STEP-2. Add GitLab Repo:

```
root@host:~# curl -sS  
https://packages.gitlab.com/install/repositories/gitlab/gitlab-  
ce/script.deb.sh | sudo bash
```

## ❑ STEP-3. Install GitLab CE on Ubuntu 22.04:

```
root@host:~# sudo EXTERNAL_URL="http://gitlab.linuxtechi.net" apt  
install gitlab-ce
```

## ❑ STEP-4. Add Firewall Rules:

```
root@host:~# sudo ufw allow http  
root@host:~# sudo ufw allow https
```

# Thank You!

Arash Foroughi  
2023-24

***SEMATEC COMPANY***

---